

LLD Learning Series - Day 3

Topics: SOLID Design Principles (Part 1) - SRP, OCP, LSP

Why SOLID Principles?

In large projects with thousands of tightly coupled classes, code quickly becomes messy and hard to maintain. Bugs become expensive, both in terms of money and engineering time. SOLID principles, proposed by Robert C. Martin in 2000, are a set of best practices that keep architecture clean and manageable.

Key Software Challenges These Solve

Maintainability: integrating new features without introducing bugs or requiring many code changes.

Readability: new developers should be able to understand a complex codebase quickly.

Bug fixing: poor design causes excessive debugging time.

The Five SOLID Principles

Letter	Principle	Definition
S	Single Responsibility Principle (SRP)	A class should have only one reason to change. It should do one thing.
O	Open/Closed Principle (OCP)	Software entities should be open for extension but closed for modification.
L	Liskov Substitution Principle (LSP)	Subclasses must be substitutable for their base classes without breaking correctness.
I	Interface Segregation Principle (ISP)	Covered in the next session.
D	Dependency Inversion Principle (DIP)	Covered in the next session.

This session covers SRP, OCP, and LSP in depth. ISP and DIP are reserved for the next session.

1. Single Responsibility Principle (SRP)

Definition: A class should have only one reason to change, or should perform exactly one responsibility.

Real-World Analogy

A TV remote controls a TV alone. If the same remote also controlled the AC and the fridge, it would complicate both its functionality and its maintenance. Mixing multiple responsibilities into one class creates multiple reasons for that class to change, which is undesirable.

Example: Shopping Cart

Initial design: a single ShoppingCart class that calculates the total price, prints the invoice, and saves data to the database. This violates SRP because it mixes calculation, presentation, and persistence in one place.

Refactored Design

Split the responsibilities into separate classes, each handling exactly one job:

Class Name	Responsibility
ShoppingCart	Calculate total price of products
CartInvoicePrinter	Print invoice
CartDBStorage	Save cart data to database

Now, changes to invoice printing or DB storage affect only their own classes, not the others.

Important Note

SRP does not mean one method per class. It means that all the methods in a class should serve the same single responsibility cohesively.

2. Open/Closed Principle (OCP)

Definition: Classes should be open for extension (you can add new features) but closed for modification (you should not need to change existing, working code).

The Problem

Suppose a project needs to support new storage types like MongoDB or file storage. Adding new methods directly inside the existing database storage class violates OCP, since it forces changes to code that already works.

Solution: Abstraction and Polymorphism

Introduce an abstract class or interface, for example DBPersistence, that declares a pure virtual method save().

Create separate subclasses for each storage type: SQLPersistence, MongoPersistence, FilePersistence.

Each subclass overrides save() with its own storage logic.

Benefits

To add a new storage method, just add a new subclass. Existing classes remain unchanged.

Client code interacts only with the abstract DBPersistence interface.

Polymorphism ensures the correct subclass's `save()` is called based on the actual object instantiated.

3. Liskov Substitution Principle (LSP)

Definition: Subclasses must be substitutable for their base classes without causing errors or requiring the client to know about subclass-specific details.

The Core Idea

If a client uses a base class type A, it should be able to seamlessly use a subclass B instead, without altering its code or causing failures.

Subclasses should extend the base class behaviour, not narrow it.

Example Scenario: Bank Accounts

Base class Account has two virtual methods: `deposit(amount)` and `withdraw(amount)`.

SavingsAccount: supports both deposit and withdraw.

CurrentAccount: supports both deposit and withdraw.

FixedDepositAccount: supports only deposit. Withdraw is not allowed.

The LSP Violation

FixedDepositAccount overrides `withdraw()` to throw an exception, since withdrawals are not allowed.

A client unaware of this might call `withdraw()` on a fixed deposit account and face a runtime error.

The client code now has to handle exceptions or check account types before calling `withdraw()`, which breaks substitutability.

Bad Workaround

Adding conditional checks for account type before calling methods creates tight coupling between client and account types, and also violates the Open/Closed Principle.

Proper Solution: Refactor the Hierarchy

Abstract Entity	Methods	Description
DepositOnlyAccount	<code>deposit()</code>	Accounts that allow only deposits, no withdraw
WithdrawableAccount	<code>deposit()</code> , <code>withdraw()</code>	Accounts supporting withdrawals (extends DepositOnlyAccount)

FixedDepositAccount derives from DepositOnlyAccount.

SavingsAccount and CurrentAccount derive from WithdrawableAccount.

The client holds two separate lists, one for each interface type, and invokes methods accordingly.

No conditional logic or exceptions are needed. Clean substitutability is maintained.

Technical Terms Used Today

Term	Definition
Tightly Coupled Classes	Classes directly dependent on each other's implementation details, making changes disruptive.
Composition (Has-A)	A class contains references to other objects, e.g. ShoppingCart has many Products.
Inheritance (Is-A)	A class derives from another to inherit and extend its features.
Polymorphism	Ability to use a derived class object via a base class reference, invoking overridden methods.
Abstract Class / Interface	Defines a contract with method declarations but no implementations, enforcing design rules.
Virtual Method	A method declared so it can be overridden in derived classes to customise behaviour dynamically.
Exception Handling	Mechanism to catch and handle runtime errors to avoid program crashes.

My Takeaways from Today

1. SRP is about cohesion, not minimalism. A class can have many methods, as long as they all serve one single responsibility.
2. OCP is achieved through abstraction and polymorphism. If adding a new feature means editing an existing class, that is a design smell.
3. LSP is the deepest of the three. A subclass that throws exceptions for inherited methods is a red flag that the hierarchy itself is wrong.
4. The FixedDepositAccount example taught me that the fix for LSP violations is not exception handling or type-checking, it is restructuring the class hierarchy itself.
5. All three principles point toward the same goal: minimizing the blast radius of any single change in the codebase.
6. Real-world analogies like the TV remote make abstract principles much easier to internalise than pure definitions.